REPOSITORY RESEARCH NOTE

New Lean proof simplifications for DiophantineConsistency

Five distinct mechanisms after a refreshed comparison with the existing proposal packets

Prepared for Vladimir Reshetnikov

26 October 2026

Main result. The strongest immediate refactor unifies two Pell ratio estimates by an explicit positive error factor. Both complete patched modules compile, all nineteen existing theorem headers in those files are preserved, and the combined source decreases from 716 to 639 physical lines. The report also develops arbitrary-premise backward-index synthesis, successor-based certificate transport, projected support separation, and a normal form for the quadratic pairing witnesses. Two initially promising mechanisms were removed after newly arrived packets proposed the same ideas.

Source pin

`7fa06d530df54d4aeb2e18027409df082234c69e`

Lean `4.32.0`; Mathlib revision

`81a5d257c8e410db227a6665ed08f64fea08e997`

This is a set of proof-refactoring proposals and checked candidates. It does not change the mathematical consistency claim or assert that the complete repository has been rebuilt.

Contents

1 Results, priorities, and the meaning of simplification 2
1.1 Measured results and validation status . . . 2

2 Scope and the novelty comparison 3
2.1 Reproducible source state . . . 3
2.2 What was inspected . . . 3
2.3 The distinctions that matter . . . 4

3 A: one Pell estimate with an explicit error factor 5
3.1 The duplication . . . 5
3.2 Why the generalized proof works . . . 5
3.3 Preserve both original interfaces . . . 5
3.4 Evidence and recommendation . . . 6

4 B: construct proof fragments for any number of premises 6
4.1 The existing reversed-history convention . . . 6
4.2 Finish the rule once . . . 7
4.3 Axiom-sensitive implementation . . . 7

5 C: prove fork certificates at successors first 8
5.1 The awkward boundary is the predecessor . . . 8
5.2 Isolate output zero once . . . 8

6 D: separate supports through a quotient homomorphism 9
6.1 A reusable mathematical statement . . . 9
6.2 The actual native client . . . 9
6.3 When to adopt it . . . 9

7 E: pairing certificates have canonical witnesses 10
7.1 Strengthen the specification . . . 10
7.2 What it buys and what it costs . . . 10

8 Validation, migration, and reproducibility 11
8.1 What the checks establish . . . 11
8.2 Recommended adoption sequence . . . 11
8.3 Archive contents . . . 12

9 Source guide 12

1 Results, priorities, and the meaning of simplification

The user asked for ideas that shorten and simplify the current Lean proofs, while avoiding ideas already in `docs/incoming`. That requirement changes the review substantially. Generic advice about `simp`, list lookup, shared sum-of-squares lemmas, DAG induction, power certificates, coefficient envelopes, and scanner folds is already well represented. This note therefore concentrates on different mathematical interfaces and on concrete clients of each interface.

The five mechanisms fall into two groups. The Pell estimate, arbitrary-premise index construction, and successor certificate transport offer measured source reductions: 77, 10, and 3 physical lines respectively, with helper overhead included. Support separation and the quadratic-pairing normal form are stronger interfaces whose first use increases total source. They should be evaluated as staged architectural ideas, rather than advertised as immediate compression.

ID	Mechanism	Concrete target and recommendation
A	Scale-parametric Pell estimate	`NativeHalfBinomial.upper_estimate_scaled` and the original `Ratio90.upper_estimate'`. Adopt first: one checked argument serves error factors 1 and 2.
B	Variadic premise assembly	`AnnotatedProofChecker.fragment0` through `fragment3`. Build an arbitrary list of premise fragments and their backward references once.
C	Successor-first certificate transport	`AnnotatedSyntaxGraphs.forkCertificate_iff`, soundness and completeness. Isolate output zero, then transport the positive case without predecessor casts.
D	Projected support differences	`NativeTemporalSupport54.clause_pair_one_zero`. Retain as an architectural candidate until multiple clients amortize the helper.
E	Canonical pairing witnesses	`QuadraticNatPairing.Certificate`. A stronger normal-form theorem supplies deterministic witnesses and uniqueness; explore before migration.

1.1 Measured results and validation status

ID	Scope counted	Physical lines	Verification
A	Both Pell estimate files	716 → 639; saves 77	Both complete modules compile; 19 original theorem headers preserved.
B	Complete proof checker	388 → 378; saves 10	Complete module compiles; eight audited theorem axiom outputs unchanged.
C	Complete syntax-graph module	169 → 166; saves 3	Complete module compiles; seven audited theorem axiom outputs unchanged.
D	Support client and new helper	82 → 125; adds 43	Both complete modules compile; original client statement preserved.
E	Stronger pairing API	No net reduction claimed	Extension imports the actual production predicate and compiles, including witness uniqueness.

The three immediate refactors together remove 90 physical lines: 1273 → 1183 at the validation pin. At the refreshed commit, proposal B instead gives 380 → 370, preserving the upstream

eight-line lookup cleanup; the net saving remains 90. Proposal D and the optional extension are not included in that saving.

The accounting includes new helper modules. A smaller client alone is not a net saving. Physical lines include comments and blank lines; byte counts use UTF-8. The archive also records nonblank counts where available. These are source measurements, not estimates of proof-term size or elaboration performance. The compilation receipts establish acceptance of particular artifacts; the recorded elapsed times are observations from a shared environment, not a controlled speed comparison.

2 Scope and the novelty comparison

2.1 Reproducible source state

The review is pinned to the commit on the title page. The requested Lean tree contains 574 Lean files: 391 modules under the main `DiophantineConsistency` module directory, one umbrella file, and 182 files under `checks`. This inventory counts files, not independent mathematical arguments. The top-level toolchain and package manifest identify Lean 4.32.0 and the exact Mathlib revision used for the checks.

The final refresh examined `main` at `6f46a5e3480a2fa2e45993b8090826f256aeaa56`. Intervening commits placed five original packets into research directories, added three new intake archives, and adopted some older proof simplifications. All four retained patches apply cleanly to that newer source state. Among their existing target files, only `AnnotatedProofChecker.lean` changed: an unrelated lookup cleanup removed eight lines, and the variadic patch preserves it. Its current-file measurement is therefore 380 → 370, versus 388 → 378 at the validation pin. Full Lean checks remain tied to the original pin; patch applicability to the refreshed tree is a separate recorded check.

2.2 What was inspected

All seven ZIP archives present in `docs/incoming` at the pin were extracted as data and inventoried: 1,990 members in total, with SHA-256 hashes. Their primary manuscripts, relevant supporting notes and prototypes were distinguished from bundled copies of the production sources. The intake README was also inspected. It directly refers to an older DiophantineConsistency simplification packet that had already been moved to a research directory; that manuscript was included too. That initial pass covered eight primary manuscripts, represented by fifteen LaTeX files because one report is split into sections. The final refresh then added all three newly arrived ZIPs: 203 additional members and three more primary manuscripts. The complete novelty comparison therefore covers ten ZIP archives, 2,193 inventoried members, and eleven primary manuscripts. The refreshed intake README and shared adoption record were also examined.

The audit groups the inspected prior proposals into 58 mechanism families, including the refreshed exclusions and nearest-prior comparisons. This is a comparison against these repository packets, not a claim of originality in the literature. Older K215 dossiers linked farther down the intake README were not exhaustively reopened; they are outside the eleven directly relevant DiophantineConsistency manuscripts. The refreshed intake README records the newer packets. The full exclusion map is included as `evidence/novelty_matrix.json` and a human-readable audit.

Key	Archive or repository location	Primary manuscript within the packet
P1	`Lean_Proof_Simplification_2026-10-26.zip`	`Lean_Proof_Simplification/report/report.tex`
P2	`ProveIt_Lean_Proof_Simplification_2026-10-26.zip`	`LeanReview/report/sections/`
P3	`diophantine-lean-simplification-2026-10-26(1).zip`	`diophantine-lean-simplification/report.tex`
P4	`diophantine-lean-simplification-2026-10-26.zip`	`diophantine-lean-simplification-2026-10-26/report.tex`
P5	`lean-proof-simplification-2026-10-26(2).zip`	`lean-proof-simplification-2026-10-26/report.tex`
P6	`lean-proof-simplification-2026-10-26.zip`	`lean-proof-simplification-2026-10-26/report.tex`
P7	`lean-proof-simplification-review-2026-10-26.zip`	`lean-proof-simplification-review/lean-proof-simplification-review.tex`
P8	`research/lean-proof-simplification-20261026/` under the project	`report.tex`; directly referenced by the intake README
P9	`diophantine-lean-additional-simplifications-2026-10-26.zip`	`diophantine-lean-additional-simplifications-2026-10-26/report.tex`
P10	`diophantine-lean-new-directions-2026-10-26.zip`	`new-lean-simplification-2026-10-26/report.tex`
P11	`diophantine-lean-new-ideas-2026-10-26.zip`	`diophantine-lean-new-ideas-2026-10-26/report.tex`

2.3 The distinctions that matter

The final refresh materially changed the recommendations. P11 already proposes the graph-depth budget for the same three `GraphSizeBounds` clients and the compositional sign-congruence relation for the same Pell clients. Both mechanisms were removed from this report's proposal list and code bundle. They remain only in the exclusion audit.

Three retained proposals need narrow descriptions of their contribution. Prior packets already propose `Produces.append`, `Produces.step`, and shared certified composition for `fragment2`/`fragment3`; proposal B adds arbitrary-premise backward-index synthesis, rather than claiming shared fragment composition itself. P11 already uses product equivalences for the quadratic pairing inverse graphs and proves canonical Euclidean division witnesses; proposal E specifically determines the quadratic square-pairing certificate's selector, slack, and maximum. Finally, the numerical obstruction at displacement 606 already exists in production; proposal D packages it as a projected support-difference theorem, rather than rediscovering that arithmetic certificate.

The Pell proposal is also distinct from prior half-radix simplification: those arguments concern exact quotient and remainder arithmetic, while proposal A concerns a real-valued Pell approximation with a positive multiplicative error allowance. No equivalent factor-parameterized Pell estimate, successor fork-certificate transport, or projected support-difference interface was found in the refreshed packets.

Two further tempting ideas were deliberately excluded in the initial pass. Combining decoder depth and quotation into one induction was already proposed, with an axiom-dependency caveat. Recasting minimum-norm saturation as a nonnegative-sum argument is too close to the existing shared sum-of-squares proposal to count as an independent contribution here.

3 A: one Pell estimate with an explicit error factor

3.1 The duplication

NativeHalfBinomial.lean contains `upper_estimate_scaled`, a substantial adaptation of `Jones1980.Ratio90.upper_estimate'` in PellRatio90.lean. Both proofs establish positivity, import the same Pell growth estimates, construct the same auxiliary quantities, bound a small multiplicative error, and cancel the same positive scale. The difference is the allowance attached to the limiting ratio.

Write

$$\xi = \frac{(U+1)^{2r}}{U^r}, \qquad \varepsilon = \frac{9}{Y(U+1)}, \qquad \delta = 2r\varepsilon.$$

The ordinary theorem assumes $\xi < Y + 1$ and proves $c/k < \xi + \frac{1}{2}$. The native theorem assumes $\xi < 2(Y+1)$ and proves $c/k < \xi + 1$. These are specializations of the same statement.

Proposition 3.1 (Scale-parametric Pell estimate)**.** *Retain the common natural-number assumptions and Pell identities of the two existing estimates: $U, Y, r \geq 1$, $J = 2r+1$, $a = Y(U+1)$, $c = \psi_{a+2}(2r+1)$, $k = \psi_{2UY^2+1}(r+1)$, $72r < U+1$, and $16 \leq Y(U+1)$. Here $\psi_A(n)$ denotes the Pell ordinate `Pell.yn` at base $A > 1$ and index n. If $\lambda \in \mathbb{R}$, $\lambda > 0$, and*

$$\xi < \lambda(Y+1),$$

then

$$\frac{c}{k} < \xi + \frac{\lambda}{2}.$$

3.2 Why the generalized proof works

The common part of the current proofs gives $k > 0$, $\varepsilon \geq 0$,

$$(1+\varepsilon)^r \leq 1 + 2r\varepsilon = 1 + \delta, \qquad c \leq k\xi(1+\delta).$$

It also proves

$$(Y+1)\delta < \frac{1}{2}.$$

The latter estimate is independent of λ: it uses the same $72r < U+1$ and $Y \geq 1$ bounds as before. Therefore

$$\xi\delta \leq \lambda(Y+1)\delta = \lambda\big((Y+1)\delta\big) < \frac{\lambda}{2}.$$

The first step needs $\delta \geq 0$; the last strict step needs $\lambda > 0$. Dividing the bound for c by positive k gives the result.

This is the useful abstraction boundary: the Pell growth argument is shared, while the allowed error factor remains a visible parameter. No stronger growth condition depending on λ is required. The generalized theorem remains a theorem about these Pell sequences over $\mathbb{R}$; the artifact does not pretend to prove a more general ordered-field statement.

3.3 Preserve both original interfaces

The candidate adds `Jones1980.Ratio90.upper_estimate_factor` to the existing lower-level file. The original `upper_estimate'` becomes the factor-1 specialization, and `NativeHalfBinomial.upper_estimate_scaled` becomes the factor-2 specialization. Both original theorem statements remain unchanged. A representative native wrapper is:

```
simpa using Jones1980.Ratio90.upper_estimate_factor
  hU hY hr hJ hA ha hP hc hk
  2 (by norm_num) hXiY h72 h16
```

The original `upper_estimate'` has a hypothesis `hYr:U^r<=Y` that its proof does not use. The generalized theorem omits it, while the compatibility wrapper retains it. This preserves callers and avoids making an unnecessary premise part of the new interface. The resulting unused-variable warning is expected and documented.

The helper belongs in the existing lower-level Pell file. Placing it in a native module and importing it back into the older Pell development would create the wrong dependency direction. This is why the patch deliberately touches one file outside the requested DiophantineConsistency directory as well as the native client.

3.4 Evidence and recommendation

The complete patched `PellRatio90.lean` and `NativeHalfBinomial.lean` both compile. All fourteen old theorem headers in the former and all five in the latter are unchanged after whitespace normalization. The original and candidate axiom audits agree for the two older estimates, the native estimate, and `source_half_binomial`: only `propext`, `Classical.choice`, and `Quot.sound` occur.

The older file grows from 472 to 478 lines. The native file shrinks from 244 to 161. Thus the full two-file change is 716 → 639, a net reduction of 77 physical lines, with the helper fully counted. The total UTF-8 size decreases by 3,654 bytes. This is the clearest first adoption: it removes a duplicated mathematical argument, preserves public contracts, and has complete file-level checks.

4 B: construct proof fragments for any number of premises

4.1 The existing reversed-history convention

In `AnnotatedProofChecker.lean`, a fragment with `Producesphp` extends every ambient history by `hp`. Running `p` and then `q` emits the history prefix `hq++hp`. The original `fragment0`, `fragment1`, `fragment2`, and `fragment3` each arrange a small number of premise fragments and calculate their parent references explicitly.

The reusable append law is prior work. The new construction is a collector for an arbitrary list of required premises, together with a lookup specification that is stable under appending any old history.

Define

$$\mathrm{PointsTo}(h, \ell, P) \iff \forall s, \quad \mathrm{map}(i \mapsto (h +\!\!+ s)[i]?)\ell = \mathrm{map}(\mathrm{some})P.$$

The universal ambient history is essential. A lookup theorem valid only for an empty history is insufficient for composing fragments inside an existing derivation.

Proposition 4.1 (Variadic collector)**.** *Suppose each judgment j in a list P has a fragment producing a history headed by j. Then there are lines p, a history h, and parent indices ℓ such that*

$$\mathrm{Produces}(p, h) \quad \textit{and} \quad \mathrm{PointsTo}(h, \ell, P).$$

Proof. Induct on the premise list. The empty case emits empty lists. For a head premise a, obtain a fragment producing $a :: t$. Recursively collect the remaining premises into history h with indices

ℓ. Execute the head fragment before the recursive one, so the combined history is $h +\!\!+ (a :: t)$. The new head index is $|h|$, and the remaining indices stay ℓ. The recursive lookup property is instantiated at ambient history $(a :: t) +\!\!+ s$. Associativity and the append-right lookup law give the desired result. □

For example, suppose three premise fragments emit histories of lengths 5, 3, and 4. Executed in premise order, they leave the prefix $h_3 +\!\!+ h_2 +\!\!+ h_1$. The parent indices, still in premise order, are $[7, 4, 0]$. No assumption says that a fragment emits just one judgment.

Premise	Own emitted history length	Index in the final prefix
First	5	$4 + 3 = 7$
Second	3	4
Third	4	0

4.2 Finish the rule once

After collecting premises, append one line containing the supplied rule and the computed parent indices. The existing line-validity interface handles the rule's own well-formedness and arithmetic conditions; the collector discharges only its premise lookups. This yields one variadic fragment theorem. The four old arity-specific statements become wrappers, retaining their exact parameter order and conclusions.

The construction stays propositional: it consumes explicitly supplied fragment witnesses by existential elimination. It does not select data from an unproved consistency statement and does not invoke `Classical.choose`. It also does not change the checker, the encoded syntax, or the meaning of a parent reference.

4.3 Axiom-sensitive implementation

A first implementation of the generic collector introduced `Quot.sound` into two small wrapper proofs through avoidable proof construction. The final implementation uses `congrArg` on the whole history, `Nat.le_refl` for the append-right bound, and direct empty/singleton membership arguments. This preserves the original axiom sets of `fragment0` and `fragment1`, rather than accepting a smaller proof solely because its type checks. The higher-arity wrappers keep their existing axiom dependencies.

The final complete `AnnotatedProofChecker.lean` compiles. The changed region decreases from 55 to 45 physical lines, from 50 to 39 nonblank lines, and from 2,643 to 2,460 UTF-8 bytes. The complete file decreases from 388 to 378 lines at the validation pin. Eight audited public theorem axiom outputs are byte-identical to baseline, including the original `fragment_exists` and downstream certificate endpoints.

Its payoff is arbitrary arity: a rule with four or more premises needs no new parent-index proof. The invariant preserves premise order in the references while reversing the emitted history blocks.

5 C: prove fork certificates at successors first

5.1 The awkward boundary is the predecessor

The fork code in `AnnotatedSyntaxGraphs.lean` has the form

$$\mathrm{forkCode}(a, b) = \mathrm{pair}(a + 1, b) + 1.$$

Its arithmetic certificate is the pairing certificate with the output shifted by one. The current interface is expressed using pred(out), so repeated reasoning must justify that the output is positive before commuting natural subtraction or predecessor with casts into $\mathbb{Z}$.

The proposed proof direction starts at out $= n+1$. At a successor, the shift cancels unconditionally. With the exact existing certificate definitions, the checked theorem is

$$\mathrm{ForkCertificate}(n + 1, a, b, t, s, m) \iff \mathrm{Certificate}(n, a + 1, b, t, s, m).$$

Its proof is a `simponly` calculation using cast-add, cast-one, and addition-subtraction cancellation. This theorem retains the same witnesses t, s, m; it does not merely prove an equivalence after existential quantification.

5.2 Isolate output zero once

From the last fork-certificate equation,

$$\mathrm{out} - 1 - m^2 - (a + 1) - tb = 0,$$

with all variables originally natural, the nonnegativity of m^2 and tb rules out output zero. The final candidate keeps this as a local argument in `forkCertificate_iff` and splits the natural output into zero and successor:

- At zero, the last certificate equation and nonnegativity give a contradiction.
- At $n + 1$, the successor transport theorem applies directly, and $\mathrm{pred}(n + 1) = n$.

The public predecessor-based `forkCertificate_iff` follows with its original statement. Soundness follows by applying pairing soundness and then `congrArgNat.succ`. Completeness rewrites the supplied output equality, obtains the original pairing witnesses, and transports the same witnesses through the successor theorem.

This is a useful local pattern whenever a positive code is represented as a successor of an arithmetic payload. Its application here is narrower than a generic codec abstraction: it specifically removes repeated predecessor-cast obligations from the existing fork certificate.

The final complete `AnnotatedSyntaxGraphs.lean` compiles. Including the new successor lemma, the changed region decreases from 37 to 34 physical lines and from 1,727 to 1,486 UTF-8 bytes. The complete file decreases from 169 to 166 lines. Seven audited theorem axiom outputs are byte-identical to baseline, including the graph degree, operation-count, witness-count, and fixed-body count endpoints.

6 D: separate supports through a quotient homomorphism

6.1 A reusable mathematical statement

Let G, H be additive commutative groups, $\pi : G \to H$ an additive homomorphism, $S \subseteq H$, and $t \in G$. Suppose the function f has support contained in the translated preimage of S:

$$\pi(x - t) \notin S \implies f(x) = 0.$$

Define

$$S - S = \{b - a : a, b \in S\}.$$

If $y = x + \delta$ and $\pi(\delta) \notin S - S$, then

$$f(x) = 0 \quad \vee \quad f(y) = 0.$$

Indeed, if both values were nonzero, both projected locations would belong to S, and their difference would be

$$\pi(y - t) - \pi(x - t) = \pi(y - x) = \pi(\delta),$$

contradicting the excluded difference. The common translation cancels in the group before any natural-number remainder reasoning is performed.

6.2 The actual native client

The checked application uses the quotient

$$\mathbb{Z}/(4831N)\mathbb{Z} \longrightarrow \mathbb{Z}/4831\mathbb{Z}$$

and the coarse support

$$S = \{j : j.\mathrm{val} \le 583 \ \vee \ j.\mathrm{val} = 1203 \ \vee \ j.\mathrm{val} = 1807\}.$$

The exact displacement certificate is $606 \notin S - S$. Applying the generic theorem to the temporal rotation produces the original conclusion of `NativeTemporalSupport54.clause_pair_one_zero`. The full temporal shift remains in the ambient cyclic group; only the support test is projected to the cell quotient.

The source already proves a natural-number `no_difference_606` obstruction. Its discovery is not new. The contribution here is the projected group interface that packages this obstruction and removes repeated translation/residue bookkeeping from prospective clients.

6.3 When to adopt it

The generic theorem, the modular certificate, and the complete native module compile. The existing public theorem header is unchanged. However, counting the helper increases the source from 82 to 125 physical lines and from 4,190 to 5,499 bytes. The candidate is therefore best retained until a second or third support-separation theorem can reuse it. If there is only this client, the existing direct proof is smaller.

This example illustrates why a useful idea need not be the first patch to merge. The interface makes the mathematical invariant clearer and may amortize well, but those future savings have not been measured and are not counted in the present results.

7 E: pairing certificates have canonical witnesses

7.1 Strengthen the specification

The arithmetic certificate in `QuadraticNatPairing.lean` uses a branch bit t, a slack s, and a maximum m. Existing soundness and completeness prove that the output is the pairing code and that some witnesses exist. A stronger specification determines the witnesses completely.

For $a, b \in \mathbb{N}$, define

$$t(a,b) = \begin{cases} 0 & a < b, \\ 1 & a \geq b, \end{cases} \qquad s(a,b) = \begin{cases} b - a - 1 & a < b, \\ a - b & a \geq b. \end{cases}$$

Then the checked normal-form statement is

$$\begin{aligned} &\mathrm{Certificate}(z, a, b, t, s, m) \\ &\quad \iff z = \mathrm{pair}(a,b) \ \wedge \ t = t(a,b) \ \wedge \ s = s(a,b) \ \wedge \ m = \max(a,b). \end{aligned}$$

The branch equations explain the result directly. At $t = 0$, the certificate is equivalent to

$$m = b, \qquad b = a + s + 1, \qquad z = b^2 + a.$$

At $t = 1$, it is equivalent to

$$m = a, \qquad a = b + s, \qquad z = a^2 + a + b.$$

The Boolean equation first forces $t \in \{0, 1\}$, after which the corresponding order relation and slack follow by natural arithmetic. At the equality boundary $a = b$, the second branch is forced and $s = 0$. In particular, $a = b = 0$ is handled without a special external assumption.

7.2 What it buys and what it costs

Soundness is the first projection of the normal form. Completeness uses the explicit triple $(t(a,b), s(a,b), \max(a,b))$. Uniqueness follows by comparing the forced coordinates of two certificates. Future inverse-graph clients can use this functional view rather than repeatedly unpacking existential witnesses.

The normal-form theorem is stronger than what current soundness and completeness require. It adds branch-normalization infrastructure, so this note does not claim an immediate net source reduction. Its recommended status is an exploratory interface with an explicit use case: adopt it when witness uniqueness or deterministic reconstruction is needed by multiple clients.

The supplied `CanonicalPairingIntegration.lean` imports and aliases the actual production `QuadraticNatPairing.Certificate`; it does not substitute a copied definition. Lean accepts the normal-form theorem, derived soundness and completeness, and witness uniqueness. The axiom audit reports only `propext`, `Classical.choice`, and `Quot.sound`. This is a checked extension, not a replacement of the entire production module.

The certificate equations and domain stay unchanged. Replacing natural witnesses by unrestricted integers would invalidate the branch and slack argument, so that generalization is not part of the proposal.

8 Validation, migration, and reproducibility

8.1 What the checks establish

Six complete candidate modules were checked: the two Pell files, the proof checker, the syntax-graph module, the support helper, and its native client. The optional pairing extension was checked against the actual production predicate. All four patches pass application checks against both the validation pin and refreshed commit `6f46a5e3480a2fa2e45993b8090826f256aeaa56`. Compilation and axiom comparisons remain tied to the original pin; the refreshed dependency graph was not rebuilt.

The retained archive distinguishes complete replacement production modules from the optional pairing extension importing the actual production predicate. Earlier copied-definition fixtures and rejected intermediate implementations are omitted. A theorem proved only for an abstract interface would establish only that interface's consequence; the retained support application also discharges its hypotheses in the concrete native module.

All 54 original public theorem and lemma headers in the five changed existing files were compared against the pinned source and preserved after whitespace normalization. New helper declarations are allowed; old hypotheses and conclusions are preserved where a production patch is supplied. The changes do not insert `sorry`, `admit`, new axioms, or native-evaluation certificates. Axiom outputs are included for the reviewed theorems and relevant endpoints. An unchanged standard axiom set is useful evidence of trust preservation, but it is not by itself proof that every declaration in the repository has been checked.

The full repository was not rebuilt. The targeted checks reuse compiled dependencies where the corresponding project source hashes match the pin, and build additional required modules from source. The package manifest and toolchain are supplied. Each proposed module's successful receipt is tied to its final source hash. No claimed line reduction depends on omitting the new helper code.

8.2 Recommended adoption sequence

1. Apply the Pell factor patch. It offers the largest demonstrated net deletion and changes only one shared mathematical argument plus two wrappers.
2. Apply the variadic fragment patch. Review the reversed-history invariant and retain the old arity-specific statements. Confirm the recorded axiom sets.
3. Review the successor-fork transport. Its invariant preserves witnesses pointwise, and its local boundary argument isolates the natural-number zero case.
4. Keep support differences and canonical pairing as staged interfaces until concrete additional clients justify their cost.

The four patches touch disjoint production files and apply independently; the archive's README describes their layout. They also passed application checks on the refreshed commit. Use the patches when working from that newer tree: copying a complete baseline candidate over it could discard the already-adopted lookup cleanup. The checked source pin is part of the result, not a suggestion to overwrite current work.

8.3 Archive contents

The ZIP contains this LaTeX source and its compiled PDF, complete candidate Lean modules, reviewable patches, the production-importing pairing extension, source and novelty manifests, line-count data, theorem-header comparisons, compilation receipts, axiom logs, and replay instructions. Baseline snapshots of the touched source files are included for comparison. The ten inspected proposal ZIPs are not duplicated; their repository paths, member identifiers, and hashes are recorded.

9 Source guide

Topic	Pinned primary source
Pell estimate	`NativeHalfBinomial.lean`; `Diophantine/Paper1980/PellRatio90.lean`
Premise fragments	`AnnotatedProofChecker.lean`
Fork certificate	`AnnotatedSyntaxGraphs.lean`
Support differences	`NativeTemporalSupport54.lean`; `NativeCleanBands54.lean`
Pairing witnesses	`QuadraticNatPairing.lean`
Toolchain	`lean-toolchain`; `lake-manifest.json`
Prior packets	`docs/incoming/README.md` and the eleven manuscript locations in Section 2

The source inventory supplies exact paths and hashes. The novelty matrix supplies archive-member line references for the earlier proposals. The independent review records mathematical boundary cases, header comparisons, and the rejected intermediate axiom drift in the variadic collector.